

MITRE eCTF 2026: Design for Hardware Security

Module (HSM)

Created by: The Lakota West High School Team
Members: Animesh Khanal, Caleb Kingan, Camron Obermeyer, Chantel McCormick, Levi Schoonover, Wencheng Xue

- MITRE eCTF 2026: Design for Hardware Security Module (HSM)..... 1
 - Introduction..... 2
 - 1. Threat Modeling..... 2
 - 1.1. Potential Threat Actors..... 2
 - 1.2 Potential Attacks..... 2
 - 2. System Description..... 3
 - 2.1. Security Requirements..... 3
 - 3. System Design..... 3
 - 3.1. System Behavior..... 3
 - 4. System Security..... 3
 - 4.1. Encryption..... 3
 - 4.2. Pin Authentication..... 4
 - 4.3. Decryption..... 4
 - 4.4. Initialize Vector Derivation..... 5
 - 4.5. Hashing..... 6
 - 5. System Defenses..... 7
 - 5.1. Input Validation..... 7
 - 5.2. Incorrect Pin Timeout..... 7
 - 5.3. Side Channel Attack Mitigation..... 7
 - 5.4. Remote Timing Attack Mitigation..... 7
 - Abbreviations..... 8

Introduction

Team Overview: Consists of six high school students. All of which have taken two or three consecutive years of cybersecurity classes and four of which have industry recognized certifications.

Competition Overview: This year, teams will create a solution for securely creating, storing, and transferring sensitive integrated circuit fabrication data between digital parties with varying levels of trust and permissions. The system must not allow unauthorized creation, disclosure, or modification of the IC design data.

1. Threat Modeling

1.1. Potential Threat Actors

As this is an HSM for a secure facility, we expect to have highly technical and well funded adversaries that are trying to attack our devices. It is imperative that our design can withstand both software and hardware based exploits, as any compromise of the HSM could lead to serious damage to the company.

1.2 Potential Attacks

According to MITRE EMB3D, our design is vulnerable to the following attacks:

- PID-11
 - TID-101 - Power consumption analysis side channel
 - TID-102 - Electromagnetic analysis side channel
 - TID-103 - Microarchitectural side channel
 - TID-105 - Hardware fault injection (control flow modification)
- PID-13
 - TID-113 - Unverified peripheral firmware loaded
 - TID-114 - Peripheral data bus interception
- PID-14
 - TID-111 - Untrusted external storage
 - TID-118 - Weak peripheral port electrical damage protection
- PID-15
 - TID-115 - Firmware/Data Extraction via Hardware Interface
 - TID-116 - Latent Privileged Access Port
 - TID-119 - Latent Hardware Debug Port Allows Memory/Code Manipulation

2. System Description

2.1. Security Requirements

- **Security Requirement 1:** An attacker should not be able to perform any file action without a validly provisioned HSM with the permissions to perform that action on files belonging to that group.
- **Security Requirement 2:** No PIN-protected action should be able to be completed by a user without prior knowledge of the PIN
- **Security Requirement 3:** An HSM should not successfully receive any file that was not generated by another valid HSM with write permissions for that group

3. System Design

3.1. System Behavior

The HSM's are designed to take input from the eCTF tools over UART. Each HSM takes the following six commands, which do the following tasks:

- List - lists all the metadata associated with the files stored on the HSM
- Read - downloads a file from an HSM, allowing it to be read by the user
- Write - writes a file to an HSM.
- Listen - places an HSM into listen mode, which waits on other dual HSM commands, such as interrogate and receive.
- Interrogate - allows the listing of the other HSMs files and metadata
- Receive - allows the sending of files between HSMs

4. System Security

4.1. Encryption

Encryption is performed using strict parameter validation and deterministic initialization to ensure effective data handling. Before performing any cryptography, the encryption routine validates all required parameters to prevent undefined behavior or memory faults. The plaintext buffer, encryption key, and ciphertext output buffer must all be present before execution may proceed.

```

FUNCTION encrypt_sym(plaintext, length, key, ciphertext):

    IF plaintext OR key OR ciphertext IS NULL:
        RETURN ERROR

    IF length == 0 OR length NOT multiple of BLOCK_SIZE:
        RETURN ERROR

    iv = derive_iv(key)

    INITIALIZE AES context

    result = AES_SetKey(context, key, KEY_SIZE, iv, ENCRYPT_MODE)

    IF result != SUCCESS:
        RETURN result

    RETURN AES_CBC_Encrypt(context, ciphertext, plaintext, length)

```

A 16-byte encryption key is enforced during initialization, resulting in AES-128 operation. Rather than requiring an externally supplied initialization vector, the implementation derives the IV from the key material. This design prevents CBC-style encryption while avoiding additional protocol complexity or IV storage requirements.

Following parameter validation, the encryption context is initialized using a key setup routine. Execution immediately terminates if the initialization function returns a non-zero result. This ensures encryption cannot proceed under an improperly configured cipher state, preventing partial encryption or corrupted output.

Once initialization succeeds, encryption proceeds using Cipher Block Chaining (CBC) mode. The plaintext buffer is processed sequentially in fixed-size blocks, where each block is combined with either the initialization vector or the previous ciphertext block before encryption.

4.2. Pin Authentication

For 5 out of the 6 commands that the HSM's respond to a pin is required, increasing the security of the devices. For example, a pin is required to read files, write files, list files on both a single device and another device.

4.3. Decryption

Decryption mirrors the encryption process and follows identical validation and initialization procedures to ensure symmetric operation between both routines. Prior to processing ciphertext, the implementation validates that all required buffers are present and properly initialized. Null

pointer checks prevent runtime instability and ensure the cryptographic engine operates only on valid memory references.

```
FUNCTION decrypt_sym(ciphertext, length, key, plaintext):  
  
    IF ciphertext OR key OR plaintext IS NULL:  
        RETURN ERROR  
  
    IF length == 0 OR length NOT multiple of BLOCK_SIZE:  
        RETURN ERROR  
  
    iv = derive_iv(key)  
  
    INITIALIZE AES context  
    result = AES_SetKey(context, key, KEY_SIZE, iv, DECRYPT_MODE)  
    IF result != SUCCESS:  
        RETURN result  
    RETURN AES_CBC_Decrypt(context, plaintext, ciphertext, length)
```

Ciphertext length validation ensures compatibility with AES block processing requirements. Because CBC mode operates exclusively on complete blocks, inputs that fail block alignment validation are rejected immediately. Padding interpretation is intentionally delegated to the calling component, simplifying the cryptographic interface and preventing ambiguity.

To maintain synchronization with encrypted data, the initialization vector is regenerated using the same deterministic derivation process used during encryption. Because both operations derive the IV identically from the key material, the decryption routine does not require external IV transmission or storage.

After successful context initialization, ciphertext blocks are processed sequentially using CBC mode. Each block is decrypted and then combined with either the initialization vector or the previous ciphertext block to reconstruct the original plaintext.

4.4. Initialize Vector Derivation

The initialization vector is generated through a deterministic derivation process designed to eliminate external IV management while preventing ECB encryption weaknesses. Rather than generating a random IV or requiring additional metadata transmission, the implementation derives the IV directly from the encryption key.

```
FUNCTION derive_iv(key):  
  
    CREATE full_hash[32 bytes]  
  
    full_hash = SHA256(key)  
  
    iv = first BLOCK_SIZE bytes of full_hash  
  
    RETURN iv
```

The supplied encryption key is hashed using SHA-256 to produce a 32-byte digest. The first block-sized segment of this digest is copied into the initialization vector buffer. This guarantees a non-zero IV while ensuring encryption and decryption routines independently reconstruct identical cipher states.

By deriving the IV internally, the system avoids additional communication overhead and simplifies external interfaces. Because IV generation remains deterministic, both encryption and decryption remain synchronized without transmitting additional cryptographic metadata between devices.

4.5. Hashing

The hashing function provides a simplified wrapper around the SHA-256 cryptographic hashing algorithm, allowing arbitrary-length data to be condensed into a fixed-length digest suitable for integrity verification or fingerprinting operations.

```
FUNCTION hash(data, length, hash_output):  
  
    IF data OR hash_output IS NULL:  
        RETURN ERROR  
  
    RETURN SHA256(data, length, hash_output)
```

Prior to execution, the function validates that both the input data buffer and output hash buffer are present. Requests containing invalid pointers terminate immediately, preventing undefined memory access during hashing operations.

Once validation succeeds, the supplied data is processed using SHA-256, producing a 32-byte digest written directly into the caller-provided output buffer. Because SHA-256 operates independently of fixed block alignment requirements, input data may be of any length without additional preprocessing.

5. System Defenses

5.1. Input Validation

Input validation was implemented to ensure only expected data is processed by the HSM. Incoming data is validated against strict length, format, and type constraints before being used by any type of routine or authentication. Buffer boundaries are verified to prevent overflow conditions, null pointers, and enumeration. Validation occurs before all security sensitive operations to eliminate modified input paths and reduce the risk of attacks.

5.2. Incorrect Pin Timeout

Incorrect PIN timeout mechanism is enforced to mitigate brute-force authentication attempts against the HSM. Each failed verification attempt triggers a fixed five second penalty delay before a response is returned and another attempt may be processed. The delay is applied deterministically and independent of input characteristics to prevent attackers from using timing differences to attack partial PIN correctness. The PIN consists of six hexadecimal characters resulting in over sixteen million possible combinations. The enforced delay limits the rate of automated guessing attempts. At a minimum of five seconds per failure, brute-force attacks would require an impractical amount of time in order to succeed.

5.3. Side Channel Attack Mitigation

Side-channel attack mitigation was implemented by introducing controlled electrical noise and signal instability across hardware paths to reduce the observable patterns during operation. Randomized voltage fluctuations were injected at regulated intervals to disrupt consistent voltage and power signatures, while timing jitter was added to interrupt predictable pin activity and signal transitions. These changes prevent attackers from correlating power consumption, electromagnetic emissions, or execution timing with internal cryptographic operations. By degrading signal clarity, the system preserves performance while increasing the difficulty of differential power analysis, fault injection, and other side-channel measurement techniques.

5.4. Remote Timing Attack Mitigation

Remote timing attack mitigation was implemented by enforcing specific execution timing across all sensitive authentication and authorization checks. The PIN comparison routine performs constant-time comparisons using randomized per-round and per-byte masking values to eliminate observable data-dependent branching. Each comparison executes a fixed number of rounds and repetitions regardless of input correctness, ensuring identical instruction paths and memory access patterns for both valid and invalid PINs.

Multiple comparison passes are executed and consensus validation is required before a result is accepted. Decision integrity is protected through redundant inverted state validation, preventing single fault manipulation authentication outcomes.

Bounded and deterministic delay mechanisms are applied after validation paths to normalize response latency. Invalid authentication attempts cause a fixed execution delay. On the other hand, successful paths include controlled noise delays to prevent distinguishable time signatures. These measures ensure attackers cannot infer authentication through response time measurement or statistical analysis.

Abbreviations

AES - Advanced Encryption Standard

CBC - Cipher Block Chaining

eCTF - Embedded Capture the Flag

HSM - Hardware Security Module

iv - Initialization Vector

PID - Property Identifier

PIN - Personal Identification Number

SHA256 - Secure Hashing Algorithm 256

TID - Threat Identifier

UART - Universal Asynchronous Reliever Transmitter